

If a subclass uses only some of the methods and properties inherited from its parents, the hierarchy is off-kilter. The unneeded methods may simply go unused or be redefined and give off exceptions

Reasons for the Problem

Someone was motivated to create inheritance between classes only by the desire to reuse the code in a superclass. But the superclass and subclass are completely different

Treatment

If inheritance makes no sense and the subclass really does have nothing in common with the superclass, eliminate inheritance in favor of Replace Inheritance with Delegation

If inheritance is appropriate, get rid of unneeded fields and methods in the subclass. Extract all fields and methods needed by the subclass from the parent class, put them in a new superclass, and set both classes to inherit from it (Extract Superclass)

Payoff

Improves code clarity and organization. You will no longer have to wonder why the Dog class is inherited from the Chair class (even though they both have 4 legs)

Refused Bequest (ارث رد شده)

علائم و نشانه‌ها (Signs and Symptoms)

زیرکلاس فقط بعضی از متدها و ویژگی‌های والد رو استفاده می‌کنه. متدهای اضافی یا استفاده نمیشن، یا بازتعریف میشن و Exception پرتاب می‌کنن.

مثال نشونه

```
public class Bird
{
    { /* پرواز */ } ()public virtual void Fly
    { /* آواز */ } ()public virtual void Sing
    { /* تخم‌گذاری */ } ()public virtual void LayEggs
}

public class Penguin : Bird
{
    ()public override void Fly
    }
    throw new Exception("پنگوئن نمی‌تونه پرواز کنه!"); // ❌
}

// Sing رو اصلاً استفاده نمی‌کنه
// فقط LayEggs رو می‌خواسته
{
```

دلایل ایجاد (Reasons for the Problem)

کسی فقط به خاطر استفاده مجدد از کد والد، ارث‌بری ایجاد کرده. ولی والد و زیرکلاس کاملاً متفاوت هستند.

درمان (Treatment)

#	تکنیک	کاربرد
۱	Replace Inheritance with Delegation	اگر ارث‌بری بی‌معنی و زیرکلاس هیچ شباهتی به والد نداره
۲	Extract Superclass	اگر ارث‌بری درسته، فیلدها و متدهای مشترک رو ببر به یه والد جدید

Payoff (دستاورد)

وضوح و سازماندهی بهتر کد. دیگه لازم نیست تعجب کنی که چرا کلاس Dog از Chair ارث‌بری کرده (فقط چون هر دو ۴ پا دارن!)

🎯 نکته طلایی

"فقط به خاطر اشتراک ظاهری، ارث‌بری نکن." — ارث‌بری یعنی **Is-A** (پنگوئن یه پرنده‌ست؟ بله. ولی پنگوئن یه صندلی نیست!)

📝 تمرین

این کد رو با درمان مناسب ریفکتور کن:

csharp

```
public class Chair
{
    public int Legs { get; set; } = 4

    public virtual void Sit
    {
        Console.WriteLine("نشستن روی صندلی");
    }

    public virtual void Move
    {
        Console.WriteLine("جابجا کردن صندلی");
    }
}

// این ارث‌بری فقط به خاطر Legs هست!
public class Dog : Chair
{
    public override void Sit
    {
        throw new Exception("سگ که صندلی نیست!");
    }
}
```

```
        ()public override void Move
        }

        Console.WriteLine("سگ راه میره"); // رفتار کاملاً متفاوت

    {

        ()public void Bark
        }

        Console.WriteLine("واق واق!");
    }
}
```

راهنمایی:

- اینجا ارث‌بری بی‌معنی به معنی **Replace Inheritance with Delegation** استفاده کن
- Dog نباید از Chair ارث‌بری کنه
- به جاش Legs رو به عنوان یه Property ساده داشته باشه

بفرست تا بررسی کنیم! 🙌